



UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

IC - UFAL

LUCAS TRINDADE DE SOUZA

PROJETO JACKUT PROGRAMAÇÃO 2

MACEIÓ/AL

2026

LUCAS TRINDADE DE SOUZA

PROJETO JACKUT PROGRAMAÇÃO 2

Trabalho solicitado pelo professor Mario
Hozano, na matéria de
PROGRAMAÇÃO 2 (COMP 372), para
compor a nota AB2.

MACEIÓ/AL

RESUMO

Este trabalho aborda o desenvolvimento da lógica de negócio de uma rede de relacionamentos denominada Jackut, implementada na linguagem Java. Desenvolvido para a disciplina de Programação 2 (COMP 372), o projeto soluciona o desafio de gerenciamento de contas, perfis dinâmicos, redes de amizades e envio de recados. A arquitetura baseia-se nos padrões Facade, Controller e Singleton, garantindo alto isolamento das regras de negócio. A confiabilidade do software é assegurada pela persistência de dados via Serialização Binária nativa (Interface Serializable), preservando o encapsulamento estrito das classes, aliada a um tratamento semântico de exceções e execução de testes de aceitação através da ferramenta EasyAccept.

SUMÁRIO

RESUMO.....	3
1. Descrição:.....	5
2. Principais componentes e Interações:.....	7
3. Padrões de projeto adotados:.....	7
3.1 Padrão Facade:.....	7
3.2 Padrão Controller (GRASP):.....	7
3.3 Padrão Singleton:.....	8

1. Descrição:

A arquitetura do projeto Jackut foi dividida em três escopos principais: Interface de Comunicação (garantida pela classe Facade), Controladores de Regra de Negócio (UsuarioController, AmizadeController, etc.) e Entidades de Domínio (Usuario, Perfil, Recado).

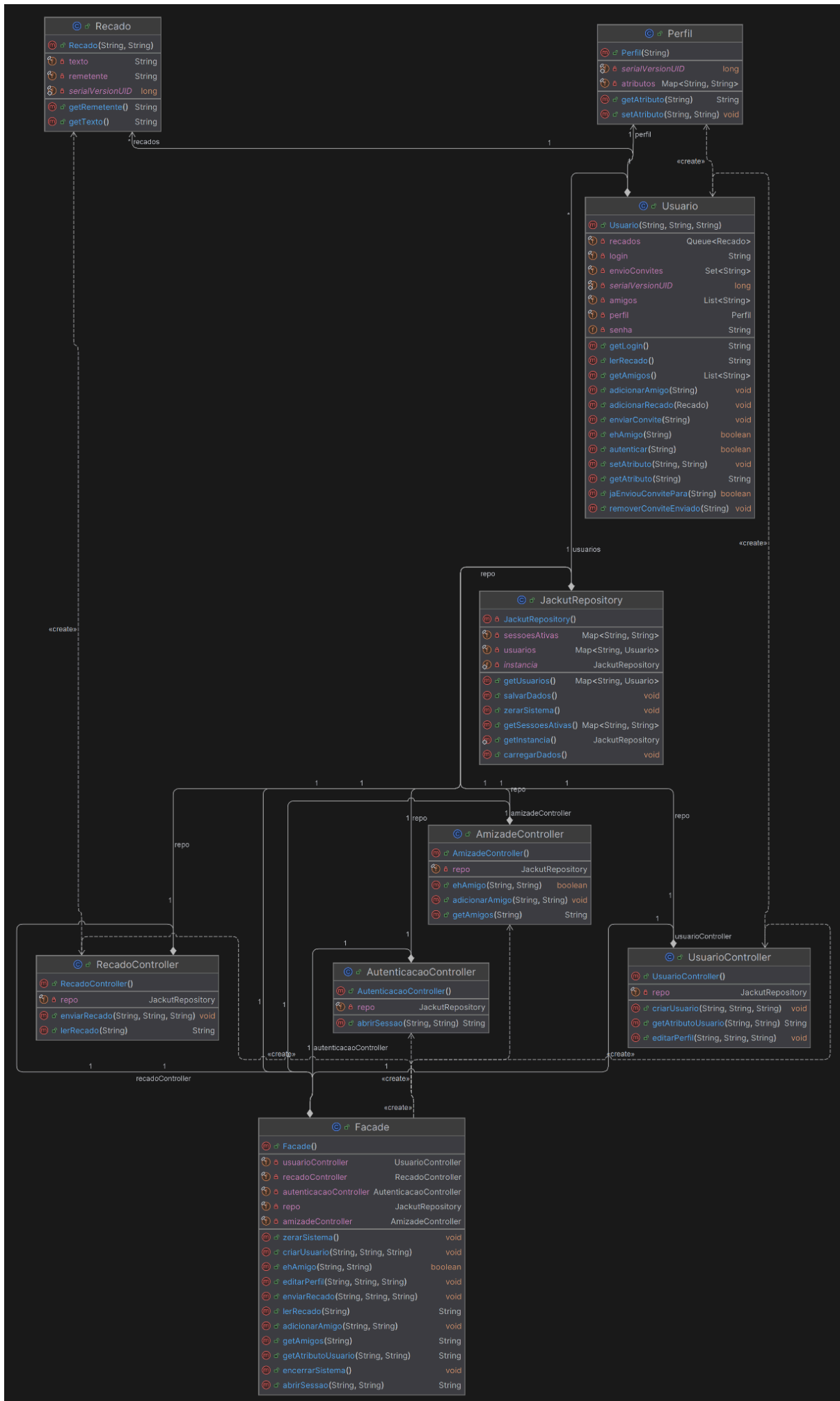
A persistência de dados foi implementada de forma local utilizando a Serialização Binária nativa do Java (ObjectOutputStream e a interface Serializable), permitindo que as Entidades mantenham seus atributos como privados e imutáveis (final).

Funcionamento da persistência e serialização: A interface Serializable atua como um marcador que sinaliza à JVM que a classe é elegível para ser convertida em um fluxo de bytes. Ao utilizar o ObjectOutputStream, o sistema mapeia o grafo de objetos da aplicação — incluindo coleções como List e Queue — e os serializa de forma hierárquica, garantindo que o estado de cada Usuario e suas amizades sejam preservados integralmente.

A escolha da serialização binária foi por ela utilizar a introspecção de tipos para restaurar os objetos diretamente, permitindo que atributos marcados como final sejam preenchidos durante o processo de desserialização, preservando a integridade e a consistência do sistema sem expor os objetos.

Para o tratamento de erros exigido pelas User Stories, foi criado um pacote centralizado (exceptions) contendo exceções personalizadas que herdam de Exception ou RuntimeException. Isso facilita a leitura do código e a resposta exata aos scripts de teste (Ex: ContaJaExisteException, AutoEnvioRecadoException).

DIAGRAMA:



2. Principais componentes e Interações:

O fluxo do sistema funciona da seguinte forma:

- **Classe Facade:** Recebe as requisições do arquivo de teste EasyAccept e atua como despachante. Ela não contém regras de negócio, apenas direciona as chamadas para os métodos adequados dos Controladores e aciona o carregamento/salvamento dos dados no repositório.
- **Classes Controllers:** As classes UsuarioController, AutenticacaoController, AmizadeController e RecadoController concentram a lógica do sistema. Elas validam regras de negócio, coordenam as chamadas aos modelos e tratam os fluxos de erro antes de retornar os resultados para a Facade.
- **JackutRepository:** Atua como o banco de dados em memória e central de persistência. Ele interage com os Controllers fornecendo acesso seguro às coleções estruturadas (HashMaps de Usuários e Sessões).
- **Modelos (Models):** São as classes que representam os dados do domínio (Usuario, Perfil, Recado). Elas operam sob encapsulamento. A classe Usuario é responsável por gerenciar e modificar suas próprias listas internas de amigos e recados.

3. Padrões de projeto adotados:

3.1 Padrão Facade:

- **Descrição Geral:** Facade é um padrão de projeto estrutural que fornece uma interface unificada e simplificada para um corpo complexo de código (um subsistema).
- **Problema Resolvido:** O sistema possui diversos gerenciadores e lógicas independentes (autenticação, amizades, perfis). Sem a Facade, o cliente de testes precisaria instanciar e coordenar todas essas classes simultaneamente.
- **Identificação da Oportunidade:** A biblioteca EasyAccept exige uma única classe de entrada para invocar os métodos mapeados nas User Stories.
- **Aplicação no Projeto:** A classe Facade foi criada para instanciar e interligar os Controllers e o Repositório. Ao invocar adicionarAmigo na Facade, ela delega a execução ao AmizadeController, blindando as camadas internas.

3.2 Padrão Controller (GRASP):

- **Descrição Geral:** O Controller é um padrão que atribui a responsabilidade de receber e coordenar as operações do sistema a uma classe mediadora, separando as entidades de dados da interface externa.
- **Problema Resolvido:** Se a classe Usuario fosse encarregada de validar senhas, buscar pessoas no banco de dados e lançar exceções de sistema, ela violaria o Princípio da Responsabilidade Única (SRP), tornando-se uma *God Class*.
- **Identificação de Oportunidade:** Era necessário isolar os fluxos de regras de negócio (como a validação de auto-relacionamento em amizades) das classes de dados puras.
- **Aplicação no projeto:** Controladores segregados por domínio. O RecadoController, por exemplo, tem a responsabilidade exclusiva de buscar o remetente e o

destinatário no repositório, aplicar a validação e, apenas em caso de sucesso, instanciar o objeto Recado para a caixa de entrada.

3.3 Padrão Singleton:

- **Descrição Geral:** Padrão criacional que garante que uma classe tenha apenas uma única instância em toda a execução do programa, fornecendo um ponto global de acesso a ela.
- **Problema Resolvido:** A fragmentação dos controladores exigia que todos acessassem a mesma base de dados em memória. A criação accidental de múltiplas instâncias de mapas de usuários geraria inconsistência de dados durante os testes.
- **Identificação de Oportunidade:** A necessidade de persistência e armazenamento em tempo de execução das sessões e usuários.
- **Aplicação no projeto:** Aplicado na classe JackutRepository através de um construtor privado e do método estático sincronizado getInstance(), garantindo que todos os Controllers manipulem as mesmas instâncias estruturais e que a gravação do arquivo binário ocorra de maneira unificada.

4. Teste da flexibilidade da arquitetura:

- **Isolamento de Atributos Dinâmicos:** A exigência de criar ou editar qualquer atributo não mapeado inicialmente (User Story 2) foi solucionada isolando essas características na classe Perfil. O uso da estrutura de dados Map<String, String> evitou uma rigidez da classe base Usuario, permitindo expansão de features sem alteração de código feito.
- **Garantia de Comportamento FIFO:** Para a User Story 4, que exige a leitura e remoção sequencial cronológica de recados, foi feito o uso de polimorfismo de interface implementando a fila de recados do usuário como uma Queue<Recado>. O uso do método estrutural nativo .poll() garantindo a precisão mecânica do FIFO de forma limpa.
- **Preservação do Encapsulamento na Persistência:** O teste de encerramento e recuperação do sistema foi superado sem degradar o design. Optar por Serialização Binária permitiu que as coleções sensíveis (como listas de amigos) permanecem imutáveis externamente (Collections.unmodifiableList) e que as classes de modelo dispensasse construtores em branco artificiais, aumentando a flexibilidade do projeto para alterações futuras.